

Assignment #1 Report: Adversarial Attacks on Neural Networks

Course: Reliable and Trustworthy Artificial Intelligence

Student: KimJaeWon

Date: April 1, 2026

0. Conclusion

1. **FGSM**
2. **PGD** FGSM
- 3.
4. MNIST CIFAR-10
5. Target Untarget

1. Overview

MNIST	CIFAR-10	CNN	FGSM(Fast Gradient Sign Method)
PGD(Projected Gradient Descent)			(Adversarial Attack)

2. Model Architecture

2.1 MNIST CNN

MNIST CNN

Input (1×28×28)

→ Conv2d(1→32, kernel=3) → ReLU → MaxPool2d(2×2)
→ Conv2d(32→64, kernel=3) → ReLU → MaxPool2d(2×2)
→ Flatten → Linear(1600→128) → ReLU → Dropout(0.5) → Linear(128→10)

- **Optimizer:** Adam (lr=0.001)
- **Loss:** CrossEntropyLoss
- **Epochs:** 10
- **Clean Test Accuracy: 99.1%**

2.2 CIFAR-10 CNN

CIFAR-10 BatchNorm CNN

Input (3×32×32)

→ Conv2d(3→32, kernel=3, pad=1) → BN → ReLU → MaxPool2d(2×2)
→ Conv2d(32→64, kernel=3, pad=1) → BN → ReLU → MaxPool2d(2×2)

→ Conv2d(64+128, kernel=3, pad=1) → BN → ReLU → MaxPool2d(2×2)
→ Flatten → Linear(2048+256) → ReLU → Dropout(0.5) → Linear(256+10)

- **Optimizer:** Adam (lr=0.001)
- **Loss:** CrossEntropyLoss
- **Epochs:** 20
- **Clean Test Accuracy: 83.4%**

3. Attack Implementation

3.1 Targeted FGSM

(target) Cross-Entropy Loss gradient step .

$$x_{\text{adv}} = x - \varepsilon \cdot \text{sign}(\nabla_x \mathcal{L}(f(x), y_{\text{target}}))$$

```
def fgsm_targeted(wrapped_model, x, target, eps):
    x_adv = x.clone().detach().requires_grad_(True)
    loss = nn.CrossEntropyLoss()(wrapped_model(x_adv), target)
    loss.backward()
    x_adv = x_adv - eps * x_adv.grad.sign()
    return x_adv.detach().clamp(0, 1)
```

3.2 Untargeted FGSM

Loss gradient step .

$$x_{\text{adv}} = x + \varepsilon \cdot \text{sign}(\nabla_x \mathcal{L}(f(x), y_{\text{true}}))$$

```
def fgsm_untargeted(wrapped_model, x, y_true, eps):
    x_adv = x.clone().detach().requires_grad_(True)
    loss = nn.CrossEntropyLoss()(wrapped_model(x_adv), y_true)
    loss.backward()
    x_adv = x_adv + eps * x_adv.grad.sign()
    return x_adv.detach().clamp(0, 1)
```

3.3 Targeted PGD

FGSM step -ball (projection) .

$$x_{\text{adv}}^{(i)} = \text{clip}_{[0,1]} \left(\text{clamp}(x_{\text{adv}}^{(i-1)} - \alpha \cdot \text{sign}(\nabla_x \mathcal{L}), x - \varepsilon, x + \varepsilon) \right)$$

```
def pgd_targeted(wrapped_model, x, target, eps, alpha=0.01, iters=40):
    x_adv = x.clone().detach()
    for _ in range(iters):
        x_adv.requires_grad_(True)
```

```

    loss = nn.CrossEntropyLoss()(wrapped_model(x_adv), target)
    loss.backward()
    x_adv = x_adv - alpha * x_adv.grad.sign()
    x_adv = torch.max(torch.min(x_adv.detach(), x + eps), x - eps).clamp(0, 1)
    return x_adv.detach()

```

3.4 Untargeted PGD

Targeted PGD , gradient step .

$$x_{\text{adv}}^{(i)} = \text{clip}_{[0,1]} \left(\text{clamp}(x_{\text{adv}}^{(i-1)} + \alpha \cdot \text{sign}(\nabla_x \mathcal{L}), x - \varepsilon, x + \varepsilon) \right)$$

4. Experimental Results

4.1 Attack Success Rate

100 . - **Targeted** : target class -
Untargeted :

MNIST (Clean Accuracy: 99.1%)

Attack Method	$\varepsilon = 0.05$	$\varepsilon = 0.10$	$\varepsilon = 0.20$	$\varepsilon = 0.30$
FGSM Targeted	38.0%	62.0%	87.0%	96.0%
FGSM Untargeted	49.0%	78.0%	94.0%	98.0%
PGD Targeted	55.0%	81.0%	95.0%	99.0%
PGD Untargeted	68.0%	89.0%	97.0%	100.0%

CIFAR-10 (Clean Accuracy: 83.4%)

Attack Method	$\varepsilon = 0.05$	$\varepsilon = 0.10$	$\varepsilon = 0.20$	$\varepsilon = 0.30$
FGSM Targeted	24.0%	47.0%	68.0%	79.0%
FGSM Untargeted	38.0%	63.0%	82.0%	91.0%
PGD Targeted	41.0%	68.0%	85.0%	93.0%
PGD Untargeted	57.0%	79.0%	93.0%	97.0%

5. Discussion

5.1

PGD > **FGSM** , **Untargeted** > **Targeted** .

- **MNIST vs CIFAR-10:** CIFAR-10 . CIFAR-10 decision boundary , $(3 \times 32 \times 32)$ gradient sign . MNIST perturbation .

5.2

(perturbation magnitude) , (imperceptibility) .

- **= 0.05:** perturbation . CIFAR-10 Targeted FGSM 24% .
- **= 0.3:** MNIST PGD Untargeted 100% , 30% .
- **= 0.1~0.2** .

5.3 Wrapped Model
